

GUÍA DE LABORATORIO N.º 9

Control de Acceso Basado en Roles

Autenticación con Laravel Breeze y autorización con spatie/laravel-permission

INF560 · Desarrollo Web Backend

Material de Laboratorio — Construcción de Aplicación Web con Laravel 13

CONTENIDO DE LA GUÍA

- 01 Configuración del proyecto Laravel 13 con PostgreSQL
- 02 Instalación y verificación de Laravel Breeze
- 03 Recurso de Notas (CRUD base) para autorizar
- 04 Instalación de spatie/laravel-permission v8
- 05 Modelo de roles y permisos mediante Seeder
- 06 Protección con el atributo #[Middleware] de Laravel 13
- 07 Directivas Blade y panel de administración de usuarios

COMPETENCIA DE LA GUÍA

El estudiante implementa un sistema de control de acceso basado en roles (RBAC) sobre una aplicación Laravel con autenticación Breeze, configurando spatie/laravel-permission para definir roles y permisos granulares, protegiendo rutas, controladores y vistas, y administrando la asignación de roles a usuarios, aplicando buenas prácticas de seguridad y autorización en el desarrollo web backend.

Materia: INF560 · Desarrollo Web Backend	Semestre: 5.º — Ing. Informática
Docente: M. Sc. Huáscar Fedor Gonzales Guzmán	Universidad: Universidad Autónoma Tomás Frías (UATF)
Proyecto: portal-rbac	Entrega (tag): v1.0.0

1 Objetivos

Al finalizar esta práctica, el estudiante será capaz de:

- Levantar un proyecto Laravel 13 con autenticación funcional usando Laravel Breeze sobre PostgreSQL.
- Instalar y configurar spatie/laravel-permission v8 dentro del stack del curso.
- Distinguir entre roles y permisos, y modelar un esquema RBAC con permisos granulares.
- Proteger rutas y controladores con el atributo #[Middleware] de Laravel 13.
- Condicionar la interfaz con directivas Blade según los permisos del usuario autenticado.

2 Prerrequisitos

- PHP 8.3 o superior y Composer 2 instalados.
- PostgreSQL 14+ en ejecución, con un usuario y permisos para crear bases de datos.
- Node.js 20+ y npm (necesarios para compilar los assets de Breeze).
- Git configurado y conocimientos de las Guías 1–8 (migraciones, Eloquent, controladores, vistas Blade y Breeze).

NOTA Versiones de referencia

Laravel 13 · PHP 8.3+ · spatie/laravel-permission ^8.0 · Laravel Breeze (instalación manual vía Composer).

spatie/laravel-permission v8 declara compatibilidad con illuminate/* ^12.0|^13.0, por lo que es la línea adecuada para Laravel 13.

3 Marco conceptual: roles y permisos

El control de acceso basado en roles (RBAC) separa la autenticación («quién eres») de la autorización («qué puedes hacer»). En lugar de codificar comprobaciones dispersas, se definen permisos atómicos que representan acciones concretas y se agrupan en roles que se asignan a los usuarios.

DEFINICIÓN Permiso vs. Rol

Permiso: la unidad mínima de autorización; describe una acción puntual (p. ej. crear-notas, editar-usuarios).

Rol: un conjunto nombrado de permisos que se asigna a uno o más usuarios (p. ej. admin, docente, estudiante).

Un usuario puede tener varios roles y, además, permisos directos. La comprobación efectiva (can) considera ambos.

spatie/laravel-permission persiste esta estructura en cinco tablas y expone una API expresiva (assignRole, givePermissionTo, hasRole, can) más middleware y directivas Blade que usaremos a lo largo de la guía.

Paso 1 Crear el proyecto y configurar PostgreSQL

Crea el proyecto seleccionando la opción «None» (sin starter kit) en el instalador, ya que instalaremos Breeze manualmente:

```
laravel new portal-rbac      # elige la opción "None" si la solicita
cd portal-rbac
```

Crea la base de datos en PostgreSQL y configura las credenciales en el archivo .env:

Terminal

```
# Crear la base de datos (desde psql o tu cliente preferido)
```

.env

```
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=portal_rbac
DB_USERNAME=postgres
DB_PASSWORD=tu_password
```

Inicializa el control de versiones:

```
git init
git add .
git commit -m "chore: proyecto base portal-rbac (Laravel 13 + PostgreSQL)"
```

Paso 2 Instalar y verificar Laravel Breeze

Breeze proporciona el andamiaje de autenticación (registro, login, recuperación de contraseña y dashboard) con vistas Blade y Tailwind. Instálalo en su variante Blade:

Terminal

```
composer require laravel/breeze --dev
php artisan breeze:install blade
php artisan migrate
npm install
npm run dev
```

Levanta el servidor y verifica que el flujo de autenticación funciona antes de continuar:

```
php artisan serve
# Visita http://127.0.0.1:8000/register y crea un usuario de prueba
```

NOTA Breeze en Laravel 13

Desde Laravel 12, Breeze se quitó del instalador laravel new, pero el paquete sigue publicándose y declara compatibilidad con Laravel 13. Por eso se crea el proyecto sin starter kit y se instala Breeze con Composer.

Paso 3 Crear el recurso Notas (CRUD base)

Las notas serán el recurso «real» que los usuarios gestionan y que luego protegeremos por permisos. Cada nota pertenece a un usuario. Genera el modelo con su migración, controlador resource y factory:

Terminal

```
php artisan make:model Nota -mcr
```

Migración

```
// database/migrations/xxxx_create_notas_table.php
public function up(): void
{
    Schema::create('notas', function (Blueprint $table) {
        $table->id();
        $table->foreignId('user_id')->constrained()->cascadeOnDelete();
        $table->string('titulo');
        $table->text('contenido');
        $table->timestamps();
    });
}
```

Modelo Nota

```
// app/Models/Nota.php
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class Nota extends Model
{
    protected $fillable = ['titulo', 'contenido'];

    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }
}
```

Relación en el modelo User

```
// app/Models/User.php (añade la relación)
use Illuminate\Database\Eloquent\Relations\HasMany;

public function notas(): HasMany
{
    return $this->hasMany(Nota::class);
}
```

Define las rutas del recurso (las protegeremos en el Paso 8) y ejecuta la migración:

```
// routes/web.php
use App\Http\Controllers\NotaController;

Route::middleware('auth')->group(function () {
    Route::resource('notas', NotaController::class);
});
```

Terminal

```
php artisan migrate
```

NOTA Vistas Blade

Crea las vistas `notas.index`, `notas.create`, `notas.edit` y `notas.show` extendiendo el layout de Breeze (x-app-layout). En esta guía nos centramos en la autorización; reutiliza el patrón de formularios y listados de las Guías anteriores.

BLOQUE B · RBAC con spatie/laravel-permission

Paso 4 Instalar spatie/laravel-permission y migrar

Instala el paquete, publica su configuración y su migración, y ejecuta las migraciones:

Terminal

```
composer require spatie/laravel-permission "^8.0"
php artisan vendor:publish --provider="Spatie\Permission\PermissionServiceProvider"
php artisan migrate
php artisan optimize:clear
```

La migración publicada crea cinco tablas que sostienen todo el modelo RBAC:

Tabla	Propósito
roles	Catálogo de roles (admin, docente, estudiante).
permissions	Catálogo de permisos atómicos (crear-notas, etc.).
model_has_roles	Roles asignados a cada usuario (relación polimórfica).
model_has_permissions	Permisos asignados directamente a un usuario.
role_has_permissions	Permisos que pertenecen a cada rol.

Paso 5 Configurar el modelo User y registrar los middleware

Añade el trait `HasRoles` al modelo `User`; este expone toda la API de roles y permisos:

```
// app/Models/User.php
use Spatie\Permission\Traits\HasRoles;

class User extends Authenticatable
{
    use HasFactory, Notifiable, HasRoles;
    // ...
}
```

Registra los alias de middleware del paquete en `bootstrap/app.php` para poder usarlos como `role`, `permission` y `role_or_permission`:

```
// bootstrap/app.php
use Spatie\Permission\Middleware\RoleMiddleware;
use Spatie\Permission\Middleware\PermissionMiddleware;
use Spatie\Permission\Middleware\RoleOrPermissionMiddleware;

->withMiddleware(function (Middleware $middleware) {
    $middleware->alias([
        'role' => RoleMiddleware::class,
        'permission' => PermissionMiddleware::class,
        'role_or_permission' => RoleOrPermissionMiddleware::class,
    ]);
});
```

ADVERTENCIA El registro de alias es obligatorio

spatie/laravel-permission ya no registra automáticamente estos alias. Si omities este paso, Laravel lanzará «Target class [role] does not exist» al proteger una ruta. En spatie v8 el namespace es Middleware (singular), no Middlewares.

Paso 6 Definir roles y permisos con un Seeder

Centraliza la creación de permisos y roles en un seeder idempotente (usa firstOrCreate y syncPermissions, de modo que pueda re-ejecutarse sin duplicar datos):

Terminal

```
php artisan make:seeder RolePermissionSeeder

// database/seeder/RolePermissionSeeder.php
namespace Database\Seeders;

use App\Models\User;
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\Hash;
use Spatie\Permission\Models\Permission;
use Spatie\Permission\Models\Role;
use Spatie\Permission\PermissionRegistrar;

class RolePermissionSeeder extends Seeder
{
    public function run(): void
    {
        // Limpiar la caché de permisos antes de sembrar
        app(PermissionRegistrar::class)->forgetCachedPermissions();

        // 1) Permisos
        $permisos = [
            'ver-usuarios', 'crear-usuarios', 'editar-usuarios',
            'eliminar-usuarios', 'gestionar-roles',
            'ver-notas', 'crear-notas', 'editar-notas',
            'eliminar-notas', 'ver-todas-las-notas',
        ];
        foreach ($permisos as $permiso) {
            Permission::firstOrCreate(['name' => $permiso]);
        }

        // 2) Roles con sus permisos
        $admin = Role::firstOrCreate(['name' => 'admin']);
        $admin->syncPermissions(Permission::all());

        $docente = Role::firstOrCreate(['name' => 'docente']);
        $docente->syncPermissions([
            'ver-usuarios',
            'ver-notas', 'crear-notas', 'editar-notas',
            'eliminar-notas', 'ver-todas-las-notas',
        ]);

        $estudiante = Role::firstOrCreate(['name' => 'estudiante']);
        $estudiante->syncPermissions([
            'ver-notas', 'crear-notas', 'editar-notas', 'eliminar-notas',
        ]);

        // 3) Usuarios
        $usuario = User::firstOrCreate(
            ['email' => 'admin@uatf.edu.bo'],
```

```

        ['name' => 'Administrador', 'password' => Hash::make('password')]
    );
    $usuario->assignRole('admin');

    $usuario = User::firstOrCreate(
        ['email' => 'huascar@uatf.edu.bo'],
        ['name' => 'Huascar Gonzales', 'password' => Hash::make('password')]
    );
    $usuario->assignRole('docente');

    $usuario = User::firstOrCreate(
        ['email' => 'juan@uatf.edu.bo'],
        ['name' => 'Juan Flores', 'password' => Hash::make('password')]
    );
    $usuario->assignRole('estudiante');
    }
}

```

Invoca el seeder desde DatabaseSeeder y ejecútalo:

```

// database/seeder/DatabaseSeeder.php
public function run(): void
{
    $this->call(RolePermissionSeeder::class);
}

Terminal
php artisan db:seed --class=RolePermissionSeeder

```

Matriz de permisos por rol

Rol	Usuarios	Notas
admin	Todo + gestionar-roles	Todo + ver-todas-las-notas
docente	ver-usuarios	CRUD + ver-todas-las-notas
estudiante	—	CRUD de sus propias notas

Paso 7 Asignar y verificar en Tinker

Comprueba la API del paquete de forma interactiva antes de cablear la interfaz:

```

php artisan tinker

>>> $u = App\Models\User::where('email','admin@uatf.edu.bo')->first();
>>> $u->getRoleNames(); // ["admin"]
>>> $u->can('ver-todas-las-notas'); // true

>>> $otro = App\Models\User::factory()->create();
>>> $otro->assignRole('estudiante');
>>> $otro->hasRole('estudiante'); // true
>>> $otro->can('ver-usuarios'); // false
>>> $otro->syncRoles(['docente']); // reemplaza sus roles
>>> $otro->can('ver-usuarios'); // true

```

DEFINICIÓN Métodos clave

assignRole / removeRole / syncRoles — gestionan roles del usuario.

givePermissionTo / revokePermissionTo — permisos directos.

hasRole / hasAnyRole — comprueban roles; can / hasPermissionTo — comprueban permisos (vía rol o directos).

Paso 8 Proteger rutas y controladores con #[Middleware]

Laravel 13 permite declarar middleware directamente sobre el controlador con el atributo repetible #[Middleware], que admite los modificadores only y except. Aplícalo al NotaController usando los middleware permission del paquete:

```
<?php

namespace App\Http\Controllers;

use App\Http\Requests\StoreNotaRequest;
use App\Http\Requests\UpdateNotaRequest;
use App\Models\Nota;
use App\Models\User;
use Illuminate\Http\RedirectResponse;
use Illuminate\Routing\Attributes\Controllers\Middleware;
use Illuminate\Support\Facades\Auth;
use Illuminate\View\View;

#[Middleware('auth')]
#[Middleware('permission:ver-notas', only: ['index', 'show'])]
#[Middleware('permission:crear-notas', only: ['create', 'store'])]
#[Middleware('permission:editar-notas', only: ['edit', 'update'])]
#[Middleware('permission:eliminar-notas', only: ['destroy'])]
class NotaController extends Controller
{
    public function index(): View
    {
        /** @var User $user */
        $user = Auth::user();

        $notas = $user->can('ver-todas-las-notas')
            ? Nota::with('user')->latest()->get()
            : $user->notas()->latest()->get();

        return view('notas.index', compact('notas'));
    }

    public function create(): View
    {
        return view('notas.create');
    }

    public function store(StoreNotaRequest $request): RedirectResponse
    {
        /** @var User $user */
        $user = Auth::user();
        $user->notas()->create($request->validated());
    }
}
```

```
        return redirect()->route('notas.index')->with('success', 'Nota creada
correctamente.');
```

```
    }

    public function show(Nota $nota): View
    {
        $this->autorizarDueno($nota);

        return view('notas.show', compact('nota'));
    }

    public function edit(Nota $nota): View
    {
        $this->autorizarDueno($nota);

        return view('notas.edit', compact('nota'));
    }

    public function update(UpdateNotaRequest $request, Nota $nota): RedirectResponse
    {
        $this->autorizarDueno($nota);

        $nota->update($request->validated());

        return redirect()->route('notas.index')->with('success', 'Nota actualizada
correctamente.');
```

```
    }

    public function destroy(Nota $nota): RedirectResponse
    {
        $this->autorizarDueno($nota);

        $nota->delete();

        return redirect()->route('notas.index')->with('success', 'Nota eliminada
correctamente.');
```

```
    }

    private function autorizarDueno(Nota $nota): void
    {
        /** @var User $user */
        $user = Auth::user();

        abort_unless(
            $nota->user_id === Auth::id()
            || $user->can('ver-todas-las-notas'),
            403
        );
    }
}
```

Paso 9 Condicionar la vista con directivas Blade

El paquete añade directivas Blade que muestran u ocultan fragmentos según roles o permisos. Úsalas en el navbar de Breeze (resources/views/layouts/navigation.blade.php) para enlazar solo lo que el usuario puede usar:

```
{{-- Enlaces condicionados por permiso --}}
@can('ver-notas')
    <x-nav-link :href="route('notas.index')"
                :active="request()->routeIs('notas.*')">
        {{ __('Notas') }}
    </x-nav-link>
@endcan
```

Dentro de notas.index, muestra el botón de crear y las acciones de editar/eliminar solo a quien corresponda:

```
@can('crear-notas')
    <a href="{{ route('notas.create') }}">Nueva nota</a>
@endcan
```

ADVERTENCIA La vista no sustituye al backend

Ocultar un botón mejora la experiencia, pero no protege nada por sí solo. La autorización real vive en el middleware y en las comprobaciones del controlador; las directivas Blade son solo la capa visual.

4 Rúbrica de evaluación

Criterio	Pts
Proyecto Laravel 13 + PostgreSQL y Breeze funcionando (registro/login).	15
Recurso Notas con migración, modelo, relaciones y CRUD operativo.	15
Instalación correcta de spatie/laravel-permission y las cinco tablas migradas.	10
Trait HasRoles y registro de los alias de middleware en bootstrap/app.php.	10
Seeder idempotente con la matriz de roles y permisos completa.	15
Protección de controladores con #[Middleware] + verificación de propiedad.	25
Directivas Blade aplicadas en el navbar y en las acciones de notas.	10
TOTAL	100

5 Entregables

- Repositorio Git del proyecto portal-rbac con historial de commits coherente.
- Etiqueta (tag) v1.0.0 marcando el estado final de la guía.
- Capturas del flujo: login, listado de notas según rol y panel de asignación de roles.

```
git add .
git commit -m "feat: RBAC con spatie/laravel-permission sobre Breeze"
git tag -a v1.0.0 -m "GL9: roles y permisos"
```

6 Referencias

spatie. Laravel-permission — Documentation. <https://spatie.be/docs/laravel-permission>

Laravel. Controllers (Middleware attributes) — Laravel 13.x. <https://laravel.com/docs/13.x/controllers>

Laravel. Authorization — Laravel 13.x. <https://laravel.com/docs/13.x/authorization>

Laravel. Starter Kits / Breeze. <https://laravel.com/docs/starter-kits>